Proyecto Intermodular 2

Sprint 2 – Proyecto 1

Instrucciones breves de ejecución/despliegue



Tamara Morillo Gregorio 2º DAMS

ÍNDICE

1. Resumen de los cambios técnicos del Sprint 2 2
2. Entidad Asignatura 2
3. Repositorio de asignaturas 3
4. Controlador de asignaturas 3
5. Consulta de asignaturas 4
8. Eliminar asignaturas 6
9. Seguridad y control de acceso 7
10. Mejoras en el cierre de sesión 8
11. Instrucciones de ejecución y despliegue 9

1. Resumen de los cambios técnicos del Sprint 2

Durante el Sprint 2 se ha ampliado la aplicación web **CatchTask (Planificador y gestor de tareas académicas)** incorporando las funcionalidades necesarias para gestionar asignaturas.

Para ello, se ha creado la **entidad Asignatura**, junto con su repositorio y su controlador, siguiendo la estructura utilizada en el Sprint 1. También se han desarrollado las páginas HTML y los formularios necesarios para **consultar, añadir, modificar y eliminar asignaturas**.

Además, se ha establecido una relación entre las asignaturas y los usuarios registrados, de manera que cada usuario pueda gestionar únicamente sus propias asignaturas. Para ello, se han añadido comprobaciones en el controlador que impiden acceder a las asignaturas de otros usuarios.

También, se ha mejorado el comportamiento de la aplicación después de cerrar sesión, evitando que el navegador muestre páginas anteriores recuperadas de la caché sin comprobar de nuevo la sesión.

Por último, se ha incorporado código JavaScript para que los mensajes de éxito desaparezcan automáticamente después de tres segundos, mejorando la presentación de los mensajes de confirmación.

Con estos cambios, **CatchTask** amplía las funcionalidades desarrolladas en el Sprint 1 y permite gestionar asignaturas, manteniendo la conexión con la base de datos MySQL.

2. Entidad Asignatura

La clase **Asignatura.java**, situada en el **paquete entity**, representa las asignaturas que los usuarios pueden guardar en **CatchTask**.

Esta clase contiene los atributos **id, nombre, descripción, curso y profesor**, que permiten almacenar la información de cada asignatura en la base de datos

Para relacionar esta clase con la base de datos, se utilizan las anotaciones **@Entity, @Id y @GeneratedValue**. La anotación @Entity indica que la clase se corresponde con una entidad que se guarda en la base de datos. @Id identifica el atributo que actúa como clave primaria y @GeneratedValue permite generar automáticamente el identificador de cada asignatura.

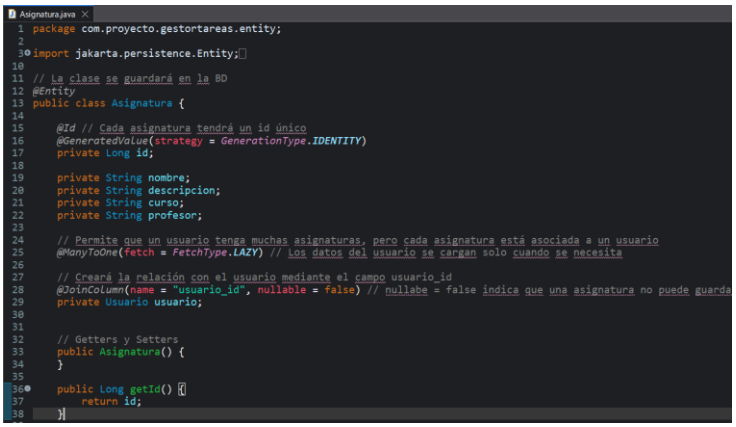


Figura 1.Implementación de la entidad Asignatura y su relación con la entidad Usuario

También, hacemos uso de la anotación @ManyToOne que sirve para relacionar cada asignatura con el usuario que la ha creado. Por último, @JoinColumn indica que esta relación se guarda mediante la columna usuario_id en la base de datos, y establece que cada asignatura debe estar asociada a un usuario.

Gracias a esta estructura, Hibernate puede relacionar los objetos de tipo Asignatura con los registros de la base de datos MySQL y facilitar el almacenamiento y la recuperación de la información.

3. Repositorio de asignaturas

La interfaz **AsignaturaRepository.java**, situada en el **paquete repository**, permite trabajar con las asignaturas guardadas en la base de datos.

Esta interfaz extiende de JpaRepository, que proporciona métodos para guardar, consultar, modificar y eliminar asignaturas sin necesidad de escribir todas las consultas manualmente.

Además, se ha creado el **método findByUsuarioId(Long usuarioId),** que permite buscar las asignaturas asociadas a un usuario concreto. De esta forma, la aplicación puede mostrar a cada usuario únicamente sus propias asignaturas.

AsignaturaRepository.java

```java
package com.proyecto.gestortareas.repository;

import java.util.List;

//Permite trabajar con las asignaturas guardadas en la base de datos
public interface AsignaturaRepository extends JpaRepository<Asignatura, Long> {

    // Busca las asignaturas que pertenecen a un usuario concreto
    List<Asignatura> findByUsuarioId(Long usuarioId);
}
```

Figura 2. Repositorio de asignaturas y método de consulta por usuario

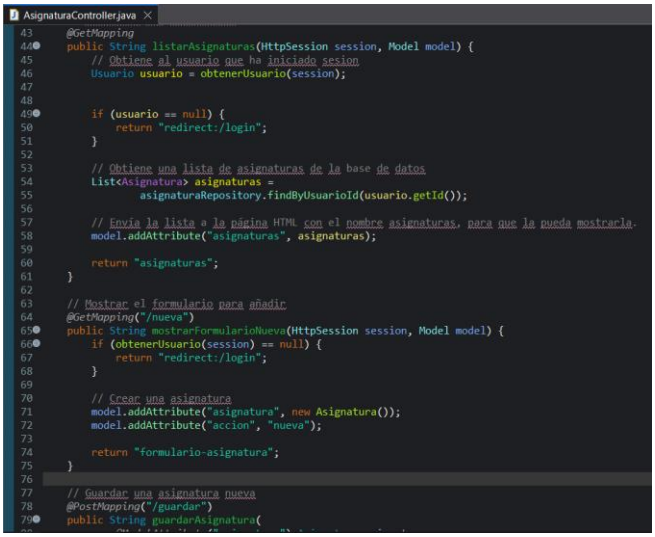
4. Controlador de asignaturas

La clase **AsignaturaController.java**, situada en el **paquete controller**, se encarga de gestionar las operaciones relacionadas con las asignaturas dentro de **CatchTask**.

Para ello, utiliza las anotaciones **@Controller y @RequestMapping**("/asignaturas"), que permiten gestionar las peticiones web relacionadas con esta sección de la aplicación.

El controlador contiene los métodos necesarios para consultar, añadir, modificar y eliminar asignaturas. Además, comprueba que el usuario haya iniciado sesión y que la asignatura que intenta modificar o eliminar le pertenezca.

Por último, **utiliza el repositorio AsignaturaRepository** para guardar y recuperar la información de la base de datos. También muestra mensajes para informar al usuario cuando una asignatura se añade, modifica o elimina correctamente.



```java
AsignaturaController.java
43      @GetMapping
44      public String listarAsignaturas(HttpSession session, Model model) {
45          // Obtiene al usuario que ha iniciado sesion
46          Usuario usuario = obtenerUsuario(session);
47
48
49          if (usuario == null) {
50              return "redirect:/login";
51          }
52
53          // Obtiene una lista de asignaturas de la base de datos
54          List<Asignatura> asignaturas =
55                  asignaturaRepository.findByUsuarioId(usuario.getId());
56
57          // Envia la lista a la página HTML con el nombre asignaturas, para que la pueda mostrarla.
58          model.addAttribute("asignaturas", asignaturas);
59
60          return "asignaturas";
61      }
62
63      // Mostrar el formulario para añadir
64      @GetMapping("/nueva")
65      public String mostrarFormularioNueva(HttpSession session, Model model) {
66          if (obtenerUsuario(session) == null) {
67              return "redirect:/login";
68          }
69
70          // Crear una asignatura
71          model.addAttribute("asignatura", new Asignatura());
72          model.addAttribute("accion", "nueva");
73
74          return "formulario-asignatura";
75      }
76
77      // Guardar una asignatura nueva
78      @PostMapping("/guardar")
79      public String guardarAsignatura(
```

Figura 3. Controlador de asignaturas y métodos de consulta y creación.

```java
@GetMapping("/editar/{id}") // id = representa el id de la asignatura
public String mostrarFormularioEditar(
        @PathVariable Long id, // recoge ese número de la dirección web.
        HttpSession session,
        Model model, // envia la asignatura a la página HTML

        // Si no encuentra asignatura, muestra mensaje
        RedirectAttributes redirectAttributes) {

    Usuario usuario = obtenerUsuario(session);

    if (usuario == null) {
        return "redirect:/login";
    }

    Asignatura asignatura = asignaturaRepository.findById(id).orElse(null);

    if (asignatura == null
            || !asignatura.getUsuario().getId().equals(usuario.getId())) {
        redirectAttributes.addFlashAttribute(
                "error", "No se ha encontrado la asignatura.");
        return "redirect:/asignaturas";
    }

    // Si la asignatura existe y pertenece al usuario, se ejecutara esto...
    model.addAttribute("asignatura", asignatura);
    model.addAttribute("accion", "editar");

    return "formulario-asignatura";
}

// Guardar los cambios de una asignatura
@PostMapping("/actualizar/{id}")
public String actualizarAsignatura(
        @PathVariable Long id,
        @ModelAttribute("asignatura") Asignatura datosFormulario,
        HttpSession session,
```

Figura 4. Métodos de edición y actualización

5. Consulta de asignaturas

La página **asignaturas.html**, situada en la **carpeta templates**, permite al usuario consultar las asignaturas que tiene guardadas en **CatchTask**.

Cuando el usuario accede a esta sección, el controlador obtiene las asignaturas asociadas a su cuenta y las envía a la página para mostrarlas. De esta forma, cada usuario puede consultar su propia lista de asignaturas.

Desde esta página también se puede acceder a las opciones para añadir nuevas asignaturas, modificarlas o eliminarlas.

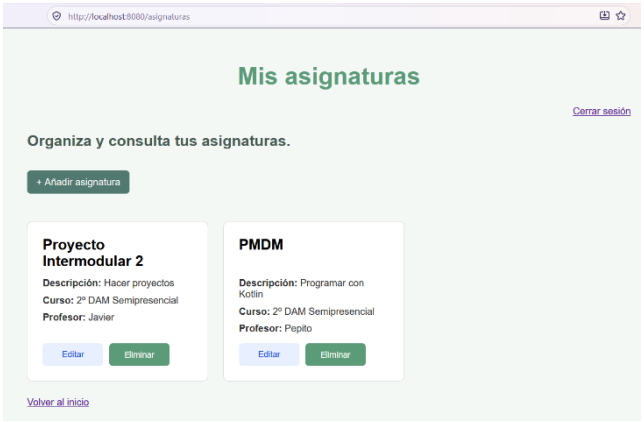


Figura 5. Página de consulta de asignaturas del usuario en CatchTask.

6. Añadir nuevas asignaturas

Para añadir asignaturas, se ha creado la página **formulario-asignatura.html**, situada en la **carpeta templates**. Esta página contiene el formulario donde el usuario introduce los datos de la asignatura, como el **nombre**, la **descripción**, el **curso y** el **profesor.**

Cuando el usuario envía el formulario, el controlador recibe los datos y los guarda en la base de datos mediante AsignaturaRepository. Además, asocia la nueva asignatura al usuario que ha iniciado sesión.

Una vez guardada, la aplicación redirige al usuario a la página de consulta de asignaturas y muestra un mensaje de confirmación.

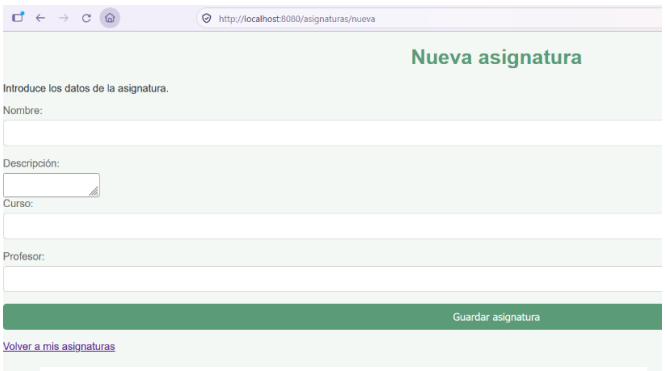


Figura 6. Formulario para añadir una nueva asignatura.

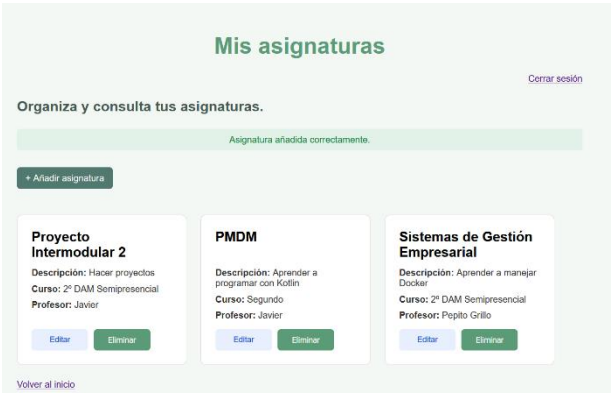


Figura 7. Confirmación de la creación de una asignatura.

7. Modificar asignaturas

CatchTask permite modificar los datos de las asignaturas que el usuario tiene guardadas. Para ello, se reutiliza la página formulario-asignatura.html, que muestra los datos actuales de la asignatura para que el usuario pueda cambiarlos.

Cuando se envía el formulario, el controlador comprueba que la asignatura existe y pertenece al usuario que ha iniciado sesión. Después, actualiza los datos y guarda los cambios en la base de datos mediante AsignaturaRepository.

Una vez finalizada la operación, la aplicación vuelve a la página de consulta de asignaturas y muestra un mensaje de confirmación.



Figura 8. Formulario de edición de una asignatura.

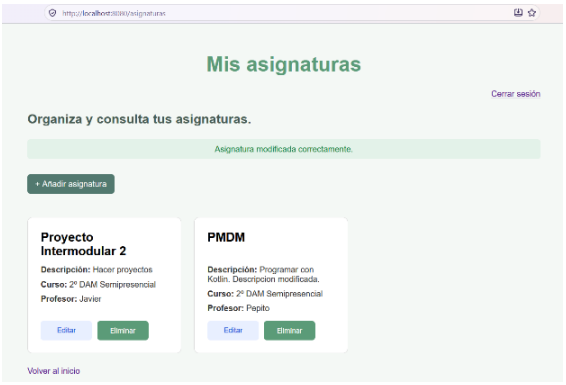


Figura 9. Asignatura modificada correctamente.

8. Eliminar asignaturas

CatchTask permite eliminar las asignaturas que el usuario ya no necesita. Para ello, desde la página asignaturas.html, el usuario puede seleccionar la opción de eliminar una asignatura.

Cuando se solicita la eliminación, el controlador comprueba que la asignatura existe y pertenece al usuario que ha iniciado sesión. Si se cumplen estas condiciones, la asignatura se elimina de la base de datos mediante el método **delete()** de **AsignaturaRepository**.

Finalmente, la aplicación vuelve a la página de consulta de asignaturas y muestra un mensaje de confirmación para informar de que la eliminación se ha realizado correctamente.

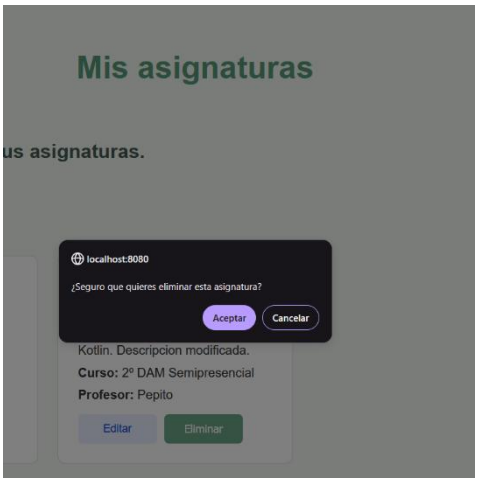


Figura 10. Opción para eliminar una asignatura.



Figura 11. Resultado de la eliminación de una asignatura.

9. Seguridad y control de acceso

Durante este Sprint se han añadido comprobaciones para garantizar que cada usuario solo pueda acceder a sus propias asignaturas.

En el controlador AsignaturaController.java, antes de realizar cualquier operación, se comprueba que el usuario haya iniciado sesión. Si no existe una sesión activa, la aplicación lo redirige a la página de inicio de sesión.

Además, antes de modificar o eliminar una asignatura, se comprueba que pertenezca al usuario actual. Si intenta acceder a una asignatura de otra cuenta, la aplicación rechaza la operación y vuelve a la lista de asignaturas.

Estas comprobaciones permiten proteger los datos de los usuarios y evitar que puedan modificar o eliminar asignaturas que pertenecen a otras cuentas.

```java
@GetMapping
public String listarAsignaturas(HttpSession session, Model model) {
    // Obtiene al usuario que ha iniciado sesion
Usuario usuario = obtenerUsuario(session);


    if (usuario == null) {
        return "redirect:/login";
    }
```

Figura 12. Comprobación de la sesión del usuario en el controlador.

```java
Asignatura asignatura = asignaturaRepository.findById(id).orElse(null);

if (asignatura == null
        || !asignatura.getUsuario().getId().equals(usuario.getId())) {
    redirectAttributes.addFlashAttribute(
            "error", "No se ha encontrado la asignatura.");
    return "redirect:/asignaturas";
}
```

Figura 13. Control de acceso a las asignaturas de cada usuario.

10. Mejoras en el cierre de sesión

Durante el Sprint 2 se ha mejorado el comportamiento de **CatchTask** después de cerrar sesión. Se detectó que, al utilizar el botón de volver del navegador, podían mostrarse páginas anteriores guardadas en la caché.

Para solucionar este problema, se ha añadido un pequeño script JavaScript en las páginas inicio.html, asignaturas.html y formulario-asignatura.html. Este código comprueba si el navegador recupera una página guardada en la caché y, en ese caso, la recarga para verificar de nuevo la sesión.

También, se ha incorporado la opción “Cerrar sesión” en la página de asignaturas, facilitando al usuario salir de su cuenta.

Con esto, se pretende mejorar el comportamiento de la aplicación al cerrar sesión.

```html
<!-- Comprueba si el navegador recupera una página guardada y la
    <script>
        window.addEventListener("pageshow", function(event) {
            if (event.persisted) {
                window.location.reload();
            }
        });
    </script>
```

Figura 14. Código para recargar las páginas recuperadas de la caché del navegador.

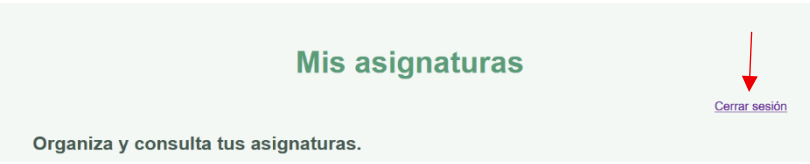


Figura 15. Opción para cerrar sesión desde la página de asignaturas.

11. Instrucciones de ejecución y despliegue

CatchTask se ha desarrollado con Java 21, Spring Boot, Maven y MySQL. Actualmente, la aplicación se ejecuta en un entorno local desde Eclipse y no está desplegada en un servidor externo.

Para ejecutar la aplicación, es necesario disponer de **Java 21, Eclipse, Maven y MySQL Workbench**, además de tener configurada la base de datos utilizada por el proyecto.

Los **pasos para iniciar la aplicación** son los siguientes:

1. Abrir el proyecto **CatchTask** en Eclipse.
2. Comprobar que el servidor MySQL está iniciado y que la base de datos está configurada correctamente.
3. Antes de arrancar el proyecto, revisar el archivo **application.properties** para verificar que los datos de conexión a MySQL son correctos.
4. Ejecutar la clase principal de Spring Boot desde Eclipse.
5. Abrir el navegador y acceder a **http://localhost:8080/login** para utilizar la aplicación.

Una vez iniciada la aplicación web, el usuario puede registrarse, iniciar sesión y gestionar sus asignaturas desde la aplicación.

El código fuente se encuentra en un repositorio de **GitHub** para facilitar su seguimiento y control de versiones. El despliegue en un servidor externo queda pendiente para una fase posterior del proyecto.